

api-gateway-load-balancing-y-service-discovery

- Clase 06 — API Gateway, Load Balancing y Service Discovery
 - 1. Contexto: el problema de acceso cuando la plataforma se distribuye
 - 2. Load Balancing: distribuir tráfico entre réplicas sanas
 - 3. API Gateway: una frontera de API, no un balanceador renombrado
 - 4. Identidad y políticas de borde: centralizar autenticación sin perder autorización
 - 5. Rate limiting y admisión de tráfico
 - 6. Service Discovery: encontrar servicios que cambian
 - 7. Diseño de referencia: el borde de una tienda online
 - 8. Trade-offs y criterios de decisión
 - 9. Casos reales
 - 10. Puntos clave
 - 11. Referencias

Clase 06 — API Gateway, Load Balancing y Service Discovery

ARQ 2026 · UCC · 16 de septiembre de 2026

1. Contexto: el problema de acceso cuando la plataforma se distribuye

Una tienda online que empezó como un único backend fue separando capacidades a medida que creció: Catálogo, Búsqueda, Carrito, Órdenes y Pagos ya son servicios independientes, cada uno con su propia base de datos y su propio ciclo de despliegue. Una sola pantalla —la vista de un producto, el resumen de una compra— necesita datos de más de uno de ellos.

Si el cliente (una app móvil, un frontend web) tuviera que saber en qué host y puerto vive cada servicio, qué ruta interna expone, cuál de sus réplicas está disponible en este momento y qué protocolo habla, la arquitectura interna se filtraría hacia afuera. Esto tiene consecuencias concretas:

- **Acoplamiento a la topología:** cualquier cambio de despliegue —mover un servicio, agregar una réplica, retirar una versión vieja— obligaría a actualizar a cada consumidor externo.
- **Lógica repetida en los clientes:** componer datos de varios servicios, reintentar una llamada fallida o aplicar una política de seguridad quedaría duplicado en cada aplicación cliente en lugar de resuelto una sola vez.

- **Exposición innecesaria:** servicios pensados para comunicarse entre sí (por ejemplo, un servicio interno de tarifas) quedarían alcanzables directamente desde internet, ampliando la superficie de ataque.
- **Políticas fragmentadas:** si cada servicio implementa su propia versión de autenticación, límites de tráfico o CORS, no hay forma de garantizar un comportamiento homogéneo en el borde.

El objetivo arquitectónico es construir una **entrada pública estable** detrás de la cual la red interna pueda cambiar de instancias, de versiones y de topología sin obligar a actualizar a cada consumidor.

Para lograrlo hace falta separar dos preguntas que suenan parecidas pero no lo son:

*¿A qué réplica sana envío esta request? — **Load Balancing** ¿Qué API expongo y bajo qué política la dejo pasar? — **API Gateway***

A eso se suma una tercera pregunta, previa a las dos anteriores: si las réplicas de un servicio nacen, se reemplazan y se retiran todo el tiempo, ¿cómo sabe el resto del sistema cuáles existen ahora mismo? Esa es la responsabilidad de **Service Discovery**.

Este documento asume conocidos HTTP, REST, JWT y los principios básicos de seguridad web (TLS, autenticación, autorización): no vuelve a explicar cómo funcionan, sino que los usa para razonar sobre **dónde** conviene ubicar cada decisión en una arquitectura distribuida.

2. Load Balancing: distribuir tráfico entre réplicas sanas

2.1 Por qué existen varias réplicas

Un servicio corre con más de una instancia por al menos tres razones: aumentar la capacidad total para atender más tráfico del que soporta un único proceso, tolerar que una instancia se caiga sin interrumpir el servicio, y permitir desplegar una versión nueva de forma gradual mientras la anterior sigue atendiendo tráfico.

Tener réplicas no alcanza por sí solo: alguien tiene que decidir, para cada conexión o request, cuál de esas réplicas la atiende. Esa es la **responsabilidad arquitectónica primaria** de un **load balancer**: elegir una instancia concreta dentro de un conjunto de réplicas equivalentes. No define por sí mismo el contrato público de una API ni las reglas de negocio; distribuye carga sobre instancias que realizan la misma capacidad.

Las herramientas reales pueden superponer capacidades: un balanceador de capa 7 puede terminar TLS o elegir un destino por host o path, y un API Gateway puede balancear tráfico hacia las réplicas de un servicio. Por eso la distinción de esta clase no se basa en una lista de funciones que una herramienta esté técnicamente impedida de hacer, sino en la pregunta que domina el diseño: **distribuir una request entre instancias equivalentes** no es lo mismo que **gobernar qué API se publica y bajo qué política**.

2.2 Algoritmos como modelos de decisión

Cada algoritmo de balanceo es un modelo con supuestos propios sobre cómo se comporta el tráfico:

Algoritmo	Cómo decide	Supuesto que asume	Cuándo distribuye mal
Round-robin	Reparte requests en secuencia fija entre las réplicas	Todas las instancias tienen capacidad similar y cada request cuesta parecido	Si una réplica es más lenta o si las requests tienen costos muy distintos, se acumula trabajo desparejo
Weighted round-robin	Igual que round-robin, pero cada réplica recibe una proporción según un peso asignado	El peso asignado refleja bien la capacidad real de cada instancia	Si la capacidad real cambia (por ejemplo, degradación parcial) y el peso no se actualiza
Least connections	Envía la próxima request a la réplica con menos conexiones activas	La cantidad de conexiones abiertas es un buen indicador de carga	Si las conexiones de larga duración no implican trabajo activo, si una request es mucho más cara que otra aunque ambas cuenten como “una conexión”, o si una réplica recién incorporada parece ociosa antes de estar preparada para recibir carga plena
Least response time	Envía la próxima request a la réplica que respondió más rápido recientemente	La latencia reciente predice la latencia inmediata siguiente	Ante variabilidad alta o picos súbitos, puede sobrecargar a la instancia que “parecía” más rápida un instante antes

Ninguno de estos algoritmos entiende el costo real de una operación de negocio: para el balanceador, una consulta liviana y una que agrega varios recursos son “una request” salvo que la métrica elegida (conexiones activas, tiempo de respuesta) refleje esa diferencia.

Una réplica nueva merece una precaución adicional: suele empezar con pocas o ninguna conexión, pero todavía puede estar calentando cachés, compilando recursos o acumulando métricas representativas. Con *least connections*, eso puede atraerle tráfico de manera desproporcionada. Un mecanismo de **slow start** incorpora la réplica gradualmente al reparto hasta que demuestre capacidad; estar marcada como lista para recibir tráfico no implica que deba recibir inmediatamente la misma proporción que una réplica estabilizada.

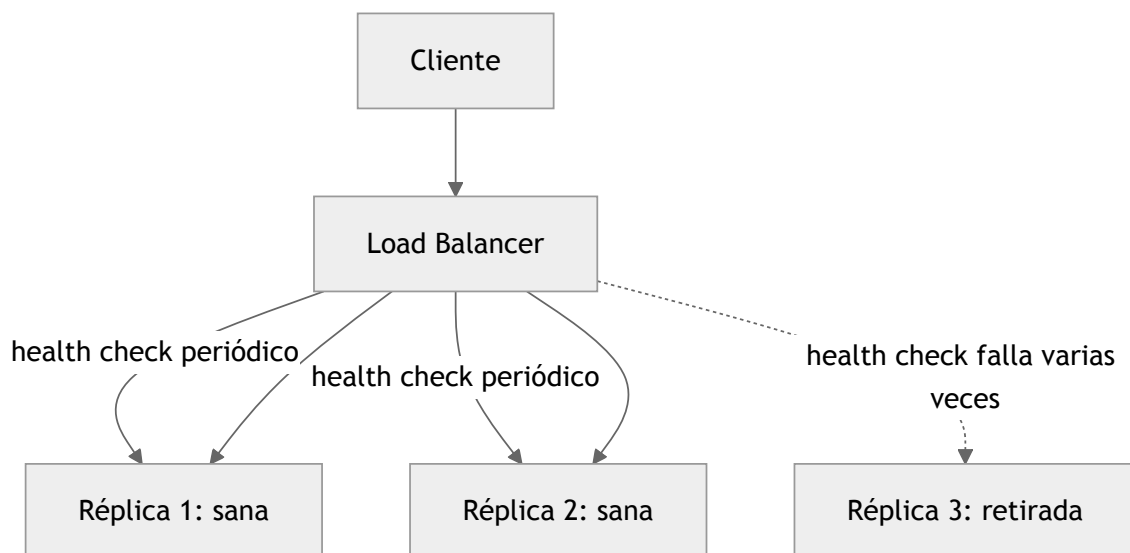
2.3 Health checks y disponibilidad efectiva

Enviar tráfico a una instancia requiere saber que puede atenderlo. Conviene distinguir tres estados que no son lo mismo:

- Un **proceso vivo**: el proceso existe y responde, pero puede no estar en condiciones de procesar tráfico real (por ejemplo, todavía está cargando datos en memoria al iniciar).
- Una **instancia lista**: superó sus verificaciones de arranque y puede recibir tráfico.

- Una **instancia con una dependencia degradada**: el proceso está vivo y listo en términos generales, pero una dependencia crítica (una base de datos, otro servicio) no responde, y por lo tanto no puede completar correctamente el trabajo que se le pediría.

Un health check bien diseñado distingue estos casos; uno pobre —por ejemplo, que solo confirma que el proceso responde en el puerto— puede declarar sana a una instancia que en realidad no puede completar ninguna operación útil, y el balanceador seguiría enviándole tráfico. Pero una readiness demasiado amplia también es dañina: no toda dependencia caída obliga a retirar una instancia si aún puede atender una parte útil y segura de sus operaciones. La pregunta es si la dependencia impide cumplir las requests que el balanceador seguirá enviándole; si no, conviene degradar esa capacidad puntual en lugar de sacar toda la réplica y concentrar aún más tráfico en las restantes. Retirar y reincorporar réplicas exige cautela: si una instancia falla una sola verificación por una demora pasajera y se la retira de inmediato, y el mismo patrón se repite apenas se reincorpora, aparece **flapping**. La instancia entra y sale del conjunto de réplicas sanas de forma inestable, generando más ruido y trabajo que el problema original. Por eso conviene exigir varias verificaciones consecutivas antes de cambiar el estado de una réplica, tanto para sacarla como para reincorporarla.



2.4 Afinidad y sesiones

Algunos diseños usan **sticky sessions**: el balanceador envía siempre las requests de una misma sesión a la misma réplica, típicamente porque esa instancia guarda estado de sesión en memoria. Esto resuelve un problema puntual, pero revela un costo: si esa réplica cae, se pierde el estado que solo ella tenía, y escalar o redistribuir tráfico se vuelve más difícil porque ya no todas las réplicas son intercambiables.

La alternativa preferida en una arquitectura distribuida es diseñar servicios sin estado y externalizar la sesión (por ejemplo, en un almacén compartido como Redis) siempre que sea posible, de modo que cualquier réplica pueda atender cualquier request sin necesitar afinidad. Redis es un ejemplo de infraestructura compartida, no una garantía automática de sesiones correctas: aún hay que definir expiración, invalidación y qué ocurre si ese almacén no está disponible.

2.5 Dónde vive un load balancer

Un balanceador puede ubicarse en más de un punto de una arquitectura, y no todos ven ni resuelven el mismo problema:

- Delante del punto de entrada público, distribuyendo tráfico entre las réplicas de ese punto de entrada.
- De forma interna, distribuyendo tráfico entre las réplicas de un servicio específico dentro de la red interna.
- Integrado al entorno de ejecución o a una librería cliente, resolviendo la elección de instancia sin un componente de red separado.

Cada ubicación resuelve una responsabilidad de balanceo distinta; ninguna de ellas resuelve, por sí sola, cómo gobernar una API pública.

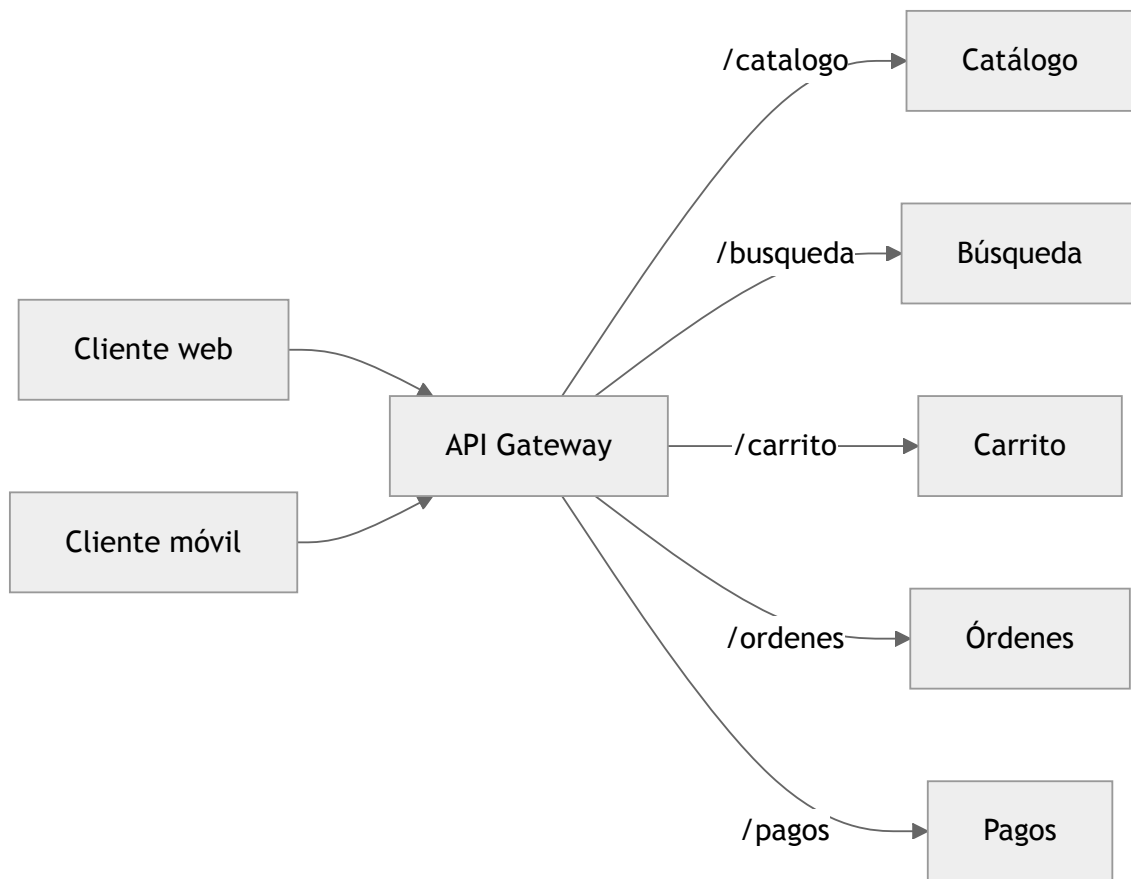
2.6 Límites de un load balancer

Un balanceador puede ofrecer algunas de estas capacidades según el producto y la capa donde opere, pero balancear tráfico **no las resuelve arquitectónicamente por sí solo**: no define la política de autenticación o autorización de una API, no modela el routing de negocio (por ejemplo, enviar una operación de pagos a Pagos y una de catálogo a Catálogo), ni diseña cuotas, composición de respuestas o compatibilidad de versiones. Esas decisiones exigen una frontera de API explícita y responsables claros.

3. API Gateway: una frontera de API, no un balanceador renombrado

3.1 Qué resuelve un API Gateway

Un **API Gateway** es el punto de entrada para clientes externos: recibe una request pública, aplica políticas de borde y decide hacia qué capacidad interna delegarla. A diferencia de un load balancer, que elige entre instancias equivalentes de un mismo servicio, el gateway resuelve un **routing semántico**: decide según host, path, método, versión de la API o audiencia del cliente, hacia qué servicio de negocio corresponde enviar la request.



3.2 Estabilidad del contrato público

El cliente conoce una URL pública y una operación de negocio —por ejemplo,

`api.tienda.../catalogo/productos/123` — pero no conoce cuántas réplicas de Catálogo existen, en qué host corren ni cómo cambiaron entre un despliegue y el siguiente. El gateway es lo que permite que ese contrato externo se mantenga estable mientras la topología interna cambia libremente.

3.3 Evolución y versionado del contrato

Mantener estable una API no significa que nunca cambie. El gateway puede hacer convivir temporalmente versiones o rutas compatibles, pero no convierte una modificación incompatible en inocua. Quitar un campo, cambiar el significado de un estado o alterar una regla de validación puede romper clientes aunque el path siga respondiendo.

La primera preferencia es una evolución compatible hacia atrás: agregar campos opcionales, conservar el significado de los existentes y comunicar una deprecación con un período de migración. Cuando una incompatibilidad es inevitable, la nueva versión debe tener un contrato explícito, consumidores identificados y una fecha de retiro verificable para la anterior. El gateway ayuda a enrutar y exponer la convivencia; la compatibilidad sigue siendo responsabilidad del equipo dueño del contrato, no un detalle que se arregla con un prefijo `/v2`.

3.4 Adaptación y composición, con límites

Un gateway puede adaptar la forma de una respuesta o combinar información de más de un servicio cuando eso resuelve una necesidad concreta de un cliente (por ejemplo, datos de Catálogo y una cotización de Entrega para la vista de producto). Lo que no debería hacer es acumular reglas de dominio, transacciones ni orquestaciones extensas: esa lógica pertenece a los servicios dueños de cada capacidad. Un gateway que crece hasta concentrar reglas de negocio deja de ser una frontera de acceso y se convierte en un monolito disfrazado de capa de borde.

3.5 Gateway, reverse proxy y BFF

Estos tres términos se usan a veces como sinónimos y no lo son:

Componente	Responsabilidad principal	Lo que no necesariamente hace
Reverse proxy	Reenvía requests hacia un destino, típicamente por reglas simples de host o path	No necesariamente gestiona identidad, cuotas ni audiencias de una API
API Gateway	Gobierna una API: identidad, cuotas, routing semántico, versión, adaptación selectiva	No sustituye la autorización de cada servicio dueño
BFF (<i>Backend for Frontend</i>)	Adapta la experiencia de una audiencia concreta (por ejemplo, la app móvil) a sus necesidades específicas	No reemplaza las políticas transversales que sirven a todas las audiencias

Un API Gateway general suele incluir capacidades de proxy, pero un proxy que solo reenvía tráfico no es, por eso, un gateway. Un gateway y un BFF pueden convivir cuando existen diferencias reales entre audiencias que lo justifican: el gateway aplica las políticas comunes (identidad, cuotas, límites de borde) y el BFF adapta el contrato final a una experiencia particular.

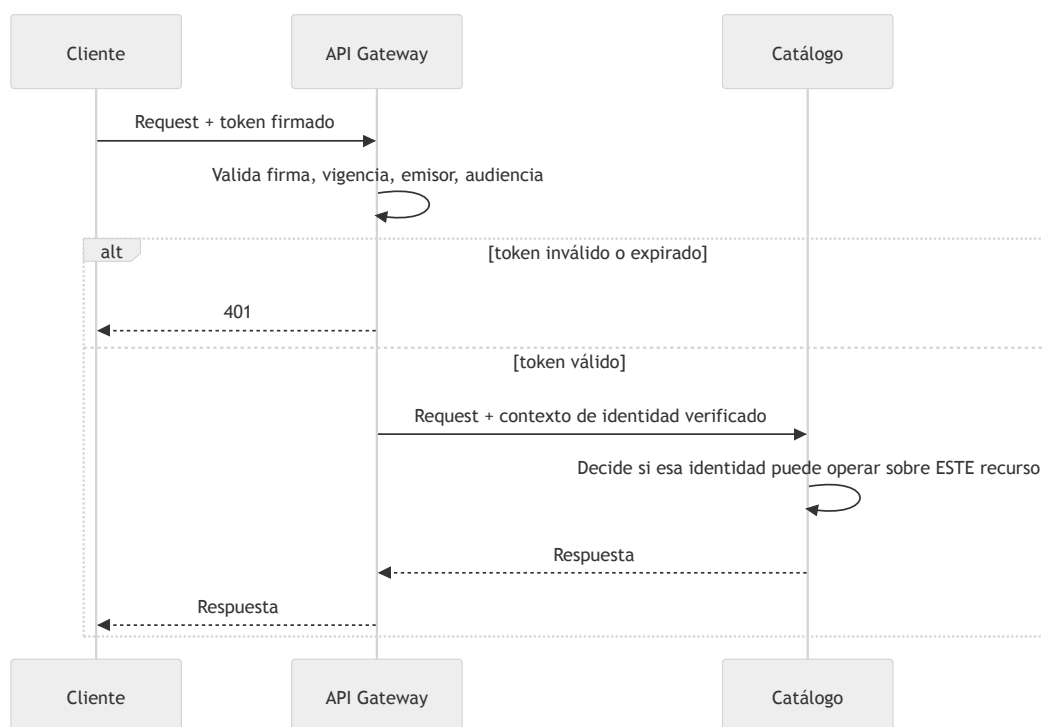
3.6 Anti-patrones

- **Gateway como monolito de negocio:** acumula reglas de dominio que deberían vivir en los servicios dueños.
- **Agregación sin límite:** compone tantas llamadas internas por cada request externa que multiplica la carga hacia los servicios de negocio sin ningún control.
- **Punto único de falla:** se despliega sin réplicas propias, convirtiéndose en el componente más frágil de toda la arquitectura que dice proteger.
- **Bypass informal:** algún cliente o integración termina llamando directamente a un servicio interno “solo por esta vez”, rompiendo la frontera que el gateway existe para sostener.

4. Identidad y políticas de borde: centralizar autenticación sin perder autorización

4.1 Punto de partida conocido

TLS protege el transporte y un token firmado permite que el gateway autentique la identidad de quien hace la request y valide sus propiedades mínimas —firma válida, vigencia no expirada, emisor esperado y audiencia correcta— antes de decidir si la deja pasar. Esto ya es conocido; lo que agrega esta sección es **dónde** conviene tomar cada una de estas decisiones en una arquitectura con varios servicios detrás de un borde común.



4.2 Autenticación centralizada, autorización distribuida

Centralizar la autenticación en el gateway tiene una ventaja clara: aplica una política homogénea y rechaza tráfico inválido lo antes posible, antes de que llegue a consumir recursos de un servicio interno. Pero autenticar no es autorizar: el gateway confirma **quién** hace la request; **qué puede hacer esa identidad sobre un recurso concreto** —leerlo, modificarlo, reservarlo, cobrarlo— es una decisión que solo el servicio dueño de ese recurso tiene el contexto de dominio para tomar. Que el gateway autentique no significa que los servicios internos dejen de proteger sus propios recursos.

4.3 Propagar identidad sin fabricar confianza

El gateway necesita transmitir hacia los servicios internos un contexto de identidad verificable, minimizando las credenciales que reenvía y evitando reenviar el token crudo cuando no hace falta. El punto crítico es que

un servicio interno debe poder distinguir **una identidad que llegó por un canal confiable del gateway de un valor que un cliente externo podría haber fabricado** (por ejemplo, un header con un ID de usuario que cualquiera podría enviar directamente).

Ese canal confiable no se presume: se diseña. Puede consistir, según el entorno, en conectividad que impide llegar al servicio sin pasar por el gateway, autenticación entre servicios y un contexto firmado o emitido por un componente confiable. El gateway debe eliminar los headers de identidad ingresados por el cliente antes de agregar los suyos. Si un servicio acepta tráfico directo o confía ciegamente en un `X-User-Id` sin verificar su procedencia, cualquiera que alcance ese servicio puede suplantar identidades y la autenticación centralizada deja de servir de nada. Aun con ese contexto confiable, el servicio conserva su autorización sobre el recurso que posee.

4.4 Políticas complementarias de borde

Además de autenticación, un gateway suele aplicar:

- **Límites de tamaño y formato de request**, para descartar tráfico malformado antes de que llegue a un servicio interno.
- **CORS** como política explícita para navegadores, no como configuración accidental.
- **Separación entre APIs públicas, privadas e internas**, declarando qué audiencia puede alcanzar cada ruta.

Estas políticas reducen la superficie de exposición del sistema, pero no reemplazan la validación de entrada ni la autorización que cada servicio debe seguir aplicando sobre sus propios datos. En particular, CORS instruye al navegador sobre qué orígenes pueden leer una respuesta; no autentica, no autoriza y no impide que un cliente fuera del navegador invoque la API.

4.5 Auditoría

Registrar la decisión de acceso y un identificador de correlación por request permite reconstruir qué pasó ante un incidente o una investigación de seguridad. Ese registro debe ser seguro: no debe incluir tokens, contraseñas ni datos personales que no sean necesarios para la traza.

5. Rate limiting y admisión de tráfico

5.1 Qué decisión resuelve

El **rate limiting** es una decisión de admisión: limita cuántas requests puede hacer un origen dado en una ventana de tiempo, para preservar la capacidad del sistema, dar un trato previsible a los distintos clientes y reducir abuso, sea accidental o malicioso.

Conviene distinguir tres controles que a veces se confunden porque protegen recursos distintos:

Control	Qué mide	Ejemplo de pregunta que responde
Rate limiting	Requests por unidad de tiempo	«¿Cuántas requests por minuto permito a este cliente?»
Cuota	Presupuesto acumulado en un período más largo	«¿Cuántas operaciones puede consumir este cliente este mes?»
Concurrencia	Cantidad de trabajo simultáneo en curso	«¿Cuántas requests de este cliente puedo estar procesando a la vez?»

5.2 Sobre qué dimensión se aplica

Un límite puede definirse por identidad autenticada, aplicación cliente, clave de API, tenant, endpoint o dirección IP. La IP es una señal auxiliar razonable —por ejemplo, para frenar tráfico anónimo antes de autenticar— pero no identifica de forma confiable a una persona: distintos usuarios pueden compartir una IP detrás de NAT o de un proxy corporativo, y un mismo atacante puede rotar de IP con facilidad.

5.3 Token bucket como modelo conceptual

Un modelo conceptual habitual es el **token bucket**: el sistema mantiene un balde con una capacidad máxima de tokens; cada request consume un token; el balde se recarga a un ritmo sostenido con el tiempo.

```

capacidad_maxima = 100 tokens
ritmo_recarga     = 10 tokens / segundo

al llegar una request:
    si tokens_disponibles >= 1:
        tokens_disponibles -= 1
        admitir request
    si no:
        rechazar request (429)

```

Esto permite un **burst** acotado —un cliente puede consumir varias requests de golpe si tenía tokens acumulados— y luego lo limita a recuperar capacidad al ritmo sostenido definido. Es cualitativamente distinto de una ventana fija (por ejemplo, “100 requests por minuto, contadas desde el minuto exacto”), que sufre un efecto de borde: un cliente puede agotar su límite justo antes de que cierre la ventana y volver a agotarlo justo después de que abra la siguiente, duplicando de hecho su tasa efectiva alrededor del corte.

5.4 Qué hacer ante el exceso

Ante una request que excede el límite, la respuesta correcta es **rechazar pronto** con un estado claro —por ejemplo, HTTP 429— y, cuando corresponda, orientación de cuándo reintentar. Si el gateway conoce una espera razonable, puede incluir el header estándar `Retry-After` con segundos o una fecha; no es obligatorio ni conviene inventar un valor cuando no hay una estimación confiable. Lo que no conviene es encolar sin límite (traslada el problema de capacidad a otro lugar sin resolverlo) ni ocultar el rechazo detrás de una respuesta que aparenta éxito.

5.5 Elegir el límite según costo y riesgo

Una búsqueda tolera una política de admisión distinta de un inicio de sesión, y ambas son distintas de una reserva o de una operación de pago. Las cuotas deben diseñarse para que un cliente legítimo no pierda una operación crítica sin que el producto le ofrezca una alternativa razonable (por ejemplo, un mensaje claro y un reintento sugerido, no un fallo silencioso).

Este documento no desarrolla qué hacer cuando la causa del problema no es un exceso de tráfico sino una dependencia lenta o caída: eso corresponde a un conjunto de mecanismos distinto (timeouts, reintentos, *circuit breakers*, *bulkheads*, degradación) que no se tratan aquí.

5.6 Un límite también se distribuye

El token bucket anterior describe un único punto de decisión. Si el gateway tiene cuatro réplicas y cada una mantiene un balde independiente de 100 tokens, un cliente que reparte tráfico entre ellas podría consumir hasta 400 tokens: el límite global dejó de ser global. Para sostener un límite compartido, las réplicas necesitan coordinar estado en un componente común o aceptar una aproximación con presupuestos locales.

El estado compartido ofrece una política más precisa, pero agrega una dependencia y latencia a cada decisión de admisión; los presupuestos locales reducen esa dependencia, pero permiten exceder el objetivo global durante una ventana. La elección depende de cuánto importa la precisión del límite y de qué ocurre si el componente que coordina contadores no está disponible. Un gateway rate-limited no elimina la necesidad de proteger recursos costosos dentro de los servicios.

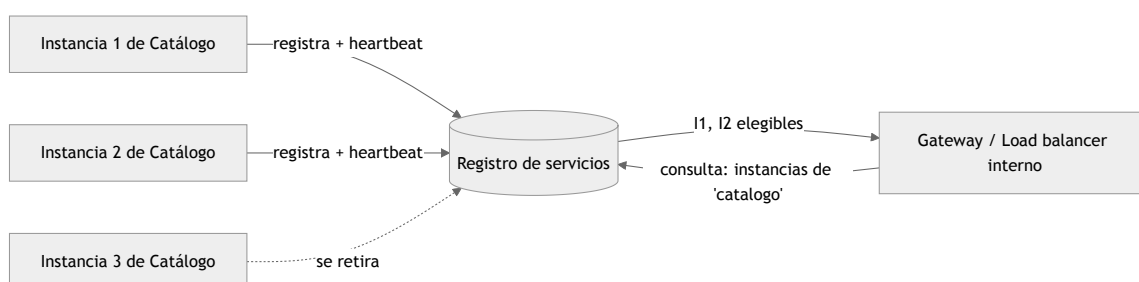
6. Service Discovery: encontrar servicios que cambian

6.1 El problema de direcciones estáticas

Las réplicas de un servicio nacen, se reemplazan, escalan y se retiran continuamente. Configurar manualmente cada host en cada cliente y en el gateway no sobrevive a una arquitectura que cambia de forma dinámica: la configuración quedaría desactualizada apenas ocurre el primer despliegue.

6.2 Registro de servicios

Un **registro de servicios** mantiene la relación entre un **nombre lógico** —por ejemplo, `catalogo`— y el conjunto de instancias disponibles para ese nombre, junto con metadatos y el estado de salud de cada una.



El registro puede completarse de dos formas: **registro propio**, donde cada instancia se anuncia al iniciar y se desanuncia al terminar (típicamente con heartbeats periódicos para confirmar que sigue viva); o **registro administrado por el entorno de ejecución**, donde una plataforma externa observa qué instancias existen y actualiza el registro sin que cada instancia tenga que anunciarse a sí misma. Ambas formas comparten los mismos riesgos: información atrasada si un heartbeat se pierde, instancias huérfanas que quedan registradas después de caerse sin avisar, y la necesidad de una política clara sobre cuánto tiempo sin heartbeat implica retirar una instancia del registro.

El registro también puede no responder en el momento de una consulta. Por eso los consumidores suelen trabajar con una lista local temporalmente cacheada: seguir usando la última lista conocida conserva disponibilidad, pero puede incluir una instancia ya retirada; descartarla de inmediato evita ese destino viejo, pero puede dejar al cliente sin ninguno. No existe una lista perfectamente actualizada gratis: el diseño decide cuánto atraso tolera, cómo expira una lista y qué señal hace retirar una instancia.

6.3 Descubrimiento client-side y server-side

Hay dos formas de usar esa información para elegir una instancia concreta:

- **Client-side:** el cliente consulta el registro (o mantiene una copia local actualizada) y decide él mismo qué instancia usar. Gana autonomía, pero traslada a cada cliente la lógica de selección y de verificación de salud.
- **Server-side:** el cliente consulta un endpoint estable —el gateway o un balanceador— que resuelve una instancia concreta apoyándose en el registro. Simplifica a los consumidores, pero concentra esa responsabilidad en la infraestructura de acceso.

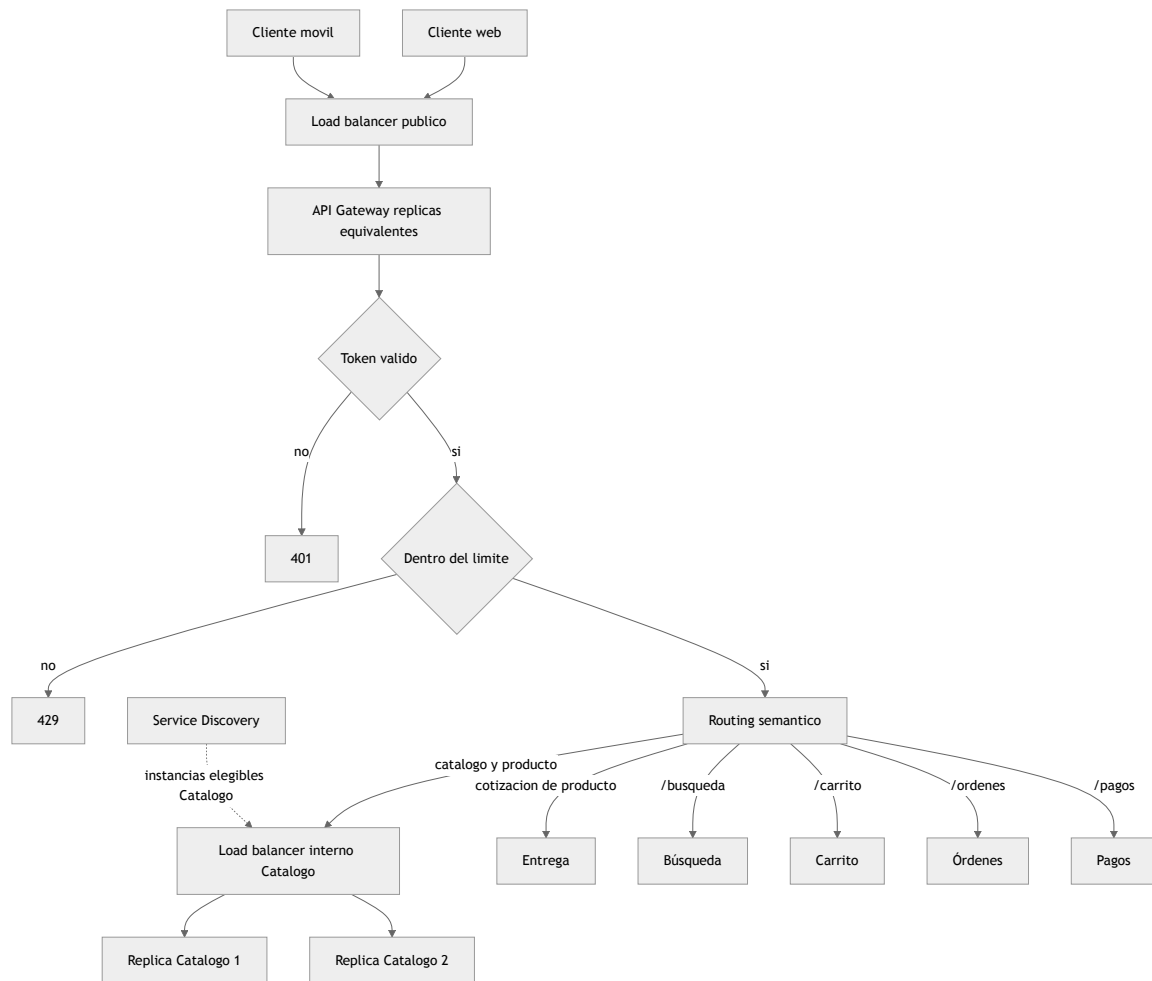
DNS, registros de servicio dedicados y plataformas de orquestación son mecanismos posibles para resolver un nombre lógico hacia instancias concretas. Lo relevante para esta materia es el **rol** que cumplen —mantener esa relación nombre-instancias actualizada— y no la configuración particular de una herramienta.

6.4 Relación con load balancing

Service Discovery y Load Balancing responden preguntas distintas y complementarias: Discovery responde **qué instancias pertenecen a un servicio y parecen elegibles ahora mismo**; el balanceador decide **cuál de esas instancias elegibles recibe esta request en particular**. Ambas decisiones necesitan la misma información de fondo —salud y cambios de membresía— pero ninguna sustituye a la otra.

7. Diseño de referencia: el borde de una tienda online

Con las piezas anteriores ya definidas, así se integran en un flujo completo:



1. El cliente web o móvil accede mediante una única URL pública. Un load balancer distribuye primero entre réplicas sanas del gateway: proteger y escalar el propio borde es, en sí mismo, un problema de balanceo.
2. El gateway autentica la request con el token, aplica el límite de admisión que corresponda a esa identidad y operación, y resuelve el routing semántico hacia Catálogo, Búsqueda, Carrito, Órdenes o Pagos.
3. Para la vista de un producto, el gateway puede componer información de Catálogo y una cotización de Entrega **solo** si eso reduce una necesidad concreta del cliente y respeta límites de llamadas internas acordados. Búsqueda no es la fuente apropiada para esa vista: su índice es un read model derivado y eventualmente consistente. El precio final, la disponibilidad y la reserva se validan siempre contra el servicio dueño en el momento de la operación real, no contra una respuesta compuesta antes.
4. El diagrama representa **server-side discovery**: el cliente externo solo conoce el gateway y **LB2** consume la lista de instancias elegibles de Catálogo que mantiene Service Discovery para elegir una réplica concreta. **LB2** puede conservar temporalmente una lista cacheada para no consultar el registro en cada request; DNS es un mecanismo posible para obtenerla, pero no el único. Ninguna réplica de Catálogo queda expuesta directamente al cliente.
5. Las rutas de administración e integración interna no se publican por defecto: se declara explícitamente qué audiencia puede alcanzar cada ruta, evitando que el gateway se convierta en un pasaje universal hacia toda la red interna.

El diseño deja ver **dos niveles de balanceo con preguntas distintas**: uno protege y escala el gateway público (la puerta de entrada), y otro distribuye las llamadas hacia las réplicas de una capacidad interna concreta (por ejemplo, Catálogo). Confundir ambos niveles —tratarlos como si fueran el mismo problema— suele llevar a configurar mal alguno de los dos.

8. Trade-offs y criterios de decisión

Decisión	Cuándo conviene	Qué evitar
Backend único sin gateway especializado	Poca complejidad, una API cohesionada, sin necesidades transversales que justifiquen otra capa	Agregar un gateway “porque es lo que se usa”, sin un problema concreto que resolver
Introducir un API Gateway	Múltiples capacidades expuestas, clientes heterogéneos, políticas de borde repetidas, necesidad de ocultar una topología dinámica	Que el gateway agregue un salto de red sin aportar un valor que lo justifique
Un load balancer alcanza, sin gateway	Varias réplicas de un servicio homogéneo con el mismo contrato, sin necesidad de routing semántico ni políticas de API adicionales	Forzar un gateway completo donde el único problema es distribuir tráfico entre instancias iguales
Introducir un BFF	Diferencias reales entre la experiencia web, móvil o de una integración de terceros que exigen contratos adaptados	Crear un BFF por moda, o duplicar en cada uno toda la lógica de dominio que debería vivir en los servicios
Service Discovery dinámico	Instancias que cambian con frecuencia por escalado, despliegues o fallas	Asumir que resuelve consistencia inmediata: la información de salud es eventual y agrega un componente más para diagnosticar

Reglas de diseño transversales: mantener servicios sin estado cuando sea posible; tratar al gateway como un componente crítico que también necesita réplicas y monitoreo; aplicar *least privilege* a credenciales y a rutas expuestas; y conservar en cada servicio, siempre, la propiedad de sus reglas de negocio y de su autorización.

9. Casos reales

9.1 Netflix: Zuul y Eureka como piezas distintas de un mismo borde

Netflix construyó **Zuul** como el servicio de borde (“front door”) por el que pasa todo el tráfico de dispositivos y sitios web hacia el backend de streaming: enruta requests, y según la documentación del proyecto habilita “dynamic routing, monitoring, resiliency, and security” para los servicios que protege. La versión Zuul 2 se reescribió sobre Netty para operar de forma asíncrona y no bloqueante, sosteniendo conexiones persistentes

a la escala de tráfico de Netflix. El Netflix Technology Blog documenta que, ya en producción, el equipo de Cloud Gateway operaba decenas de clústeres de Zuul 2 enrutando tráfico hacia alrededor de cien clústeres de servicios backend. [Netflix TechBlog — Zuul 2: The Netflix Journey to Asynchronous, Non-Blocking Systems](#); [GitHub — Netflix/zuul](#).

Eureka, en cambio, resuelve un problema distinto: es el registro donde cada servicio se anuncia y desde donde otros componentes descubren qué instancias existen en cada momento. Netflix combinó ese registro con **Ribbon**, una librería que agregaba balanceo del lado del cliente y otras capacidades de resiliencia sobre las instancias que Eureka reportaba como disponibles. El propio blog de Netflix presentó Ribbon explícitamente como el complemento que se apoya en Eureka para resolver qué instancias existen y elegir entre ellas. [Netflix TechBlog — Announcing Ribbon: Tying the Netflix Mid-Tier Services Together](#).

El caso ilustra con claridad la distinción de este documento: Zuul gobierna la entrada pública de la API; Eureka (con Ribbon) resuelve qué instancias existen y cómo elegir entre ellas. Son responsabilidades que conviven, pero ninguna sustituye a la otra.

9.2 Uber: un gateway que evoluciona junto con la organización

Uber describe en su blog de ingeniería la evolución de su API Gateway a través de varias generaciones de arquitectura, cada una enfrentando desafíos y responsabilidades distintas a medida que la compañía y sus clientes (apps de pasajeros, conductores, socios) crecieron en diversidad. El objetivo declarado de estas capas fue ofrecer un único punto de entrada para todas las apps y accesos a los servicios internos, con un lugar centralizado para enrutamiento, conversión de protocolo, *rate limiting*, *load shedding*, enriquecimiento de headers, reglas de afinidad por centro de datos, auditoría de seguridad y bloqueo de accesos. [Uber Blog — The Architecture of Uber's API Gateway](#); [Uber Blog — Designing Edge Gateway, Uber's API Lifecycle Management Platform](#).

El caso muestra que un gateway no es una decisión que se toma una sola vez: a medida que cambian la cantidad de clientes, sus necesidades y la organización que lo mantiene, las responsabilidades concentradas en el borde también evolucionan, y requieren revisarse activamente para no derivar en los anti-patrones descritos en la sección 3.6.

9.3 Kubernetes: qué mide un health check y por qué importa distinguir tipos

Kubernetes documenta oficialmente tres tipos de verificación distintos sobre un contenedor en ejecución: una **liveness probe**, que determina si el proceso sigue funcionando y provoca un reinicio si falla; una **readiness probe**, que determina si el contenedor está en condiciones de recibir tráfico y, si falla, lo excluye del balanceo sin reiniciarlo; y una **startup probe**, pensada para procesos que tardan en inicializarse, que evita marcar como no vivo a un proceso que simplemente todavía está arrancando. Cuando la readiness probe falla, la documentación es explícita: el contenedor deja de recibir tráfico de los servicios que lo balancean, sin que eso implique necesariamente un reinicio. [Kubernetes — Liveness, Readiness, and Startup Probes](#); [Kubernetes — Configure Liveness, Readiness and Startup Probes](#).

El caso confirma, con una plataforma de orquestación ampliamente documentada y no con un proveedor puntual, la distinción presentada en la sección 2.3: “proceso vivo” y “listo para recibir tráfico” son preguntas distintas, y una plataforma de referencia las modela como verificaciones separadas justamente porque combinarlas en una sola llevaría a decisiones de balanceo equivocadas.

10. Puntos clave

- Un load balancer distribuye tráfico entre réplicas equivalentes; un API Gateway gobierna la entrada y el contrato de las APIs. Pueden integrarse, pero no son equivalentes.
- Las réplicas, los health checks y el balanceo hacen que una capacidad atienda más tráfico y sobreviva a fallas de instancias individuales; no la vuelven, por sí solos, una API segura o estable.
- El gateway oculta detalles de topología y centraliza políticas transversales, pero no debe capturar la lógica de negocio de los servicios que hay detrás.
- Autenticar en el gateway reduce exposición; el contexto de identidad que llega a un servicio debe provenir de un canal verificable, y autorizar el acceso a un recurso concreto permanece en el servicio dueño de ese dominio.
- Rate limiting administra la admisión de tráfico; el criterio, la identidad usada para medirlo y la respuesta ante un exceso son decisiones de producto, capacidad y seguridad, no solo un parámetro técnico.
- Un rate limit global debe coordinarse entre las réplicas del gateway o aceptar explícitamente una aproximación; multiplicar baldes locales multiplica el límite efectivo.
- Service Discovery mantiene la relación entre un nombre lógico y sus instancias disponibles; el balanceador usa esa información para elegir un destino concreto entre las instancias elegibles.
- La salud que importa para el borde es la capacidad de atender la operación correctamente, no solamente que el proceso siga en ejecución.
- BFF, gateway, reverse proxy y load balancer pueden convivir, pero sus responsabilidades deben quedar explícitas antes de sumarlos a una arquitectura.
- El borde de acceso es un componente crítico del sistema: requiere sus propias réplicas, rutas limitadas por audiencia y credenciales de mínimo privilegio.

11. Referencias

- [Netflix TechBlog — Zuul 2: The Netflix Journey to Asynchronous, Non-Blocking Systems.](#)
- [Netflix TechBlog — Announcing Ribbon: Tying the Netflix Mid-Tier Services Together.](#)
- [GitHub — Netflix/zuul.](#)
- [Uber Blog — The Architecture of Uber's API Gateway.](#)
- [Uber Blog — Designing Edge Gateway, Uber's API Lifecycle Management Platform.](#)
- [Kubernetes — Liveness, Readiness, and Startup Probes.](#)
- [Kubernetes — Configure Liveness, Readiness and Startup Probes.](#)
- [IETF — RFC 6585: 429 Too Many Requests.](#)